

OUTILS FORMELS POUR L'INFORMATIQUE

Dr Konaté

ingngolo@gmail.com

February, 2021

Contents

LIST OF FIGURES	iii
1 ELEMENTS DE LOGIQUES	1
1.1 Proposition logiques	1
1.1.1 Regle de logique formelle	1
1.1.2 Les connecteurs logiques	1
1.1.3 Table de verite	3
1.2 Quantificateurs	3
1.2.1 Quantificateur universel	3
1.2.2 Quantificateur existentiel	3
1.3 Methode de raisonnement	4
1.3.1 Méthode de raisonnement direct	4
1.3.2 Raisonnement par recurrence	4
1.4 Induction et reccurence	5
1.4.1 Induction	5
1.4.2 Recursion	6
2 RECURSIVITES ET INDUCTIONS	8
2.1 Methode de raisonnement	8
2.1.1 Methode de raisonnement direct	8
2.1.2 Raisonnement par recurrence sur $\mathbb{N}$	8
2.2 Inductions et recurrences	9
2.2.1 Principe general d'une fonction inductive	9
2.2.2 Preuve par Induction	10
3 MOTS ET LANGAGES	12
3.1 Definition preliminaires	12
3.2 Expressions régulière et langages associés	14
4 AUTOMATES	17
4.1 Automates finis deterministes AFD	17
4.2 Automates finis non-deterministes(AFND)	18
5 Travaux Dirigés	26
5.1 Exercice I: Logiques et Fonctions booléenes	26
5.2 Exercice II: Automates	26

Introduction

Dans les années 50, *NoamChomsky* a développé un modèle mathématique pour décrire les langages formels. Deux théories découlent de ce modèle :

- **la théorie des langages formels** qui sera d'un grand intérêt pour l'informatique
- **la transformation linguistique**

La description du langage formel a été mise en œuvre par **Chomsky**. La théorie des langages formels découle de l'étude du langage naturel et sa description est appelée une grammaire. Un langage formel peut servir de modèle mathématique pour un langage naturel, tandis qu'une grammaire formelle peut servir de modèle pour une théorie linguistique.

Chapter 1

ELEMENTS DE LOGIQUES

Introduction

1.1 Proposition logiques

1.1.1 Regle de logique formelle

Definition 1. *Une proposition est une expression mathématique a laquelle on peut attribuer la table de verité vrai ou faux*

- Tout nombre premier est paire
- $\sqrt{2}$ est un nombre irrationnel
- 2 est inférieur a 4

Definition 2. *Toute proposition démontrée vrai est appelée théoreme*

Definition 3. *Soit P une proposition, la negation de P est une proposition désignant le contraire qu'on note (non p), ou bien $\bar{P}$, On peut trouver aussi la notation $\neg P$*

- Soit $E \neq 0$, $P : (a \in E)$, alors $\bar{P} : (a \in E)$
- P : La fonction f est positive, alors $\bar{P}$: la fonction P est négative

1.1.2 Les connecteurs logiques

Definition 4. *La conjonction est le connecteur logique $\langle\langle \text{et} \rangle\rangle$, $\langle\langle \wedge \rangle\rangle$. La proposition $(P \text{ et } Q)$ ou $(P \wedge Q)$ est la conjonction des deux propositions P, Q*

- $(P \wedge Q)$ est vraie si P et Q le sont toute les deux.
- $(P \wedge Q)$ est fausse si l'une ou l'autre des propositions l'est.

Definition 5. La disjonction est un connecteur logique $\langle\langle \text{ou} \rangle\rangle$, $\langle\langle \vee \rangle\rangle$. On note la conjonction $(P \text{ ou } Q)$, $(P \vee Q)$.

La proposition $P \vee Q$ est fausse si P et Q sont fausses toutes les deux, sinon $(P \vee Q)$ est vrai

Definition 6. L'implication de deux propositions P, Q est notée : $P \implies Q$.

On dit P implique Q ou bien si P alors Q . $P \implies Q$ est fausse si P est vraie et Q est fausse, sinon $(P \implies Q)$ est vraie dans les autres cas

Definition 7. La réciproque de l'implication $(P \implies Q)$ est une implication $(Q \implies P)$

- La réciproque de : (Il pleut, alors je prends mon parapluie), est : (je prends mon parapluie, alors il pleut).
- La réciproque de : (Omar a gagné au loto $\implies$ Omar a joué au loto), est : (Omar a joué au loto $\implies$ Omar a gagné au loto).
- La réciproque de : $0 \leq x \leq 9 \implies \sqrt{x} \leq 3$, est $\sqrt{x} \leq 3 \implies 0 \leq x \leq 9$

Definition 8. La contraposée de l'implication: Soit P, Q deux propositions, la contraposée de $(P \implies Q)$ est $(\bar{Q} \implies \bar{P})$, on a: $(P \implies Q)$ équivaut à $(\bar{Q} \implies \bar{P})$

- La contraposée de : (Il pleut, alors je prends mon parapluie), est (je ne prends pas mon parapluie, alors il ne pleut pas).
- La contraposée de : (Omar a gagné au loto $\implies$ Omar a joué au loto), est : (Omar n'a pas joué au loto $\implies$ Omar n'a pas gagné au loto).

Definition 9. La négation d'une proposition. Soit P, Q deux propositions on a:

$\overline{(P \implies Q)}$ équivaut à $(P \wedge \bar{Q})$

- La négation de : (il pleut, alors je prends mon parapluie), est : (il pleut et je ne prends pas mon parapluie).
- La négation de : (Omar a gagné au loto $\implies$ Omar a joué au loto), est : (Omar a gagné au loto et Omar n'a pas joué au loto).

- $(x \in [0, 1]) \implies x \geq 0$ sa negation: $(x \in [0, 1] \wedge x < 0)$

Conclusion

- La negation de $(P \implies Q)$ est $(P \wedge \overline{Q})$
- La contraposée de $(P \implies Q)$ est $(\overline{P} \implies \overline{Q})$
- La reciproque de $(P \implies Q)$ est $(Q \implies P)$

1.1.3 Table de verite

P	Q	$\neg$	$P \wedge Q$	$P \vee Q$	$P \implies Q$	$P \Leftrightarrow Q$
V	V		V	V	V	V
V	F		F	V	F	F
F	V		F	V	V	F
F	F		F	F	V	V

1.2 Quantificateurs

1.2.1 Quantificateur universel

Definition 10. Le quantificateur universel $\forall$ La relation pour tous x tel que $P(x)$ est notée : $\forall, P(x)$ se lit quel que soit x , $P(x)$.

$P(x)$: La fonction f est nulle pour tous $x \in \mathbb{R}$ devient:

$$P(x) : \forall x \in \mathbb{R}, f(x) = 0$$

1.2.2 Quantificateur existentiel

Definition 11. Le quantificateur existentiel $\exists$ la relation il existe un x tel que $P(x)$ est notée : $\exists x, P(x)$

$P(x)$: La fonction f s'annule en x_o devient:

$$P(x) : \exists x_o \in \mathbb{R}, f(x_o) = 0$$

1.3 Méthode de raisonnement

Pour montrer que $P \implies Q$ est vrai, on peut utiliser plusieurs principes de raisonnement. Emprunté à la logique mathématique, ces principes interviennent dans l'élaboration d'algorithmes divers.

1.3.1 Méthode de raisonnement direct

Principe: On suppose que P est vrai et on démontre que Q l'est aussi

Exemple: Montrons que $\forall n \in \mathbb{N}$ n est pair $\implies n^2$ est pair.

Resolution: On suppose que n est pair ie $\exists k \in \mathbb{Z}$:

$$n = 2k \text{ donc } n.n = 2.(2k^2) \implies n^2 = 2.k'$$

On pose $k' = 2k^2 \in \mathbb{Z}$ ainsi $\exists k \in \mathbb{Z}$ tel que $n^2 = 2k'$

D'où $n^2 = 2k'$ avec n pair

1.3.2 Raisonnement par récurrence

Theorem 1. Pour montrer que $P(n): \forall n \in \mathbb{Z}, n > n_0, P_n(x)$ est vrai, on suit les étapes suivantes:

- i. On montre que $P(n_0)$ est vrai (valeur initiale)
- ii. On suppose que $P(n)$ est vraie à l'ordre n
- iii. On démontre que $P(n+1)$ est vraie à l'ordre $n+1$ Alors P est vraie $\forall n > n_0$

Exemple: Montrer que $\forall n \in \mathbb{N}^*, 1 + 2 + \dots + n = \frac{n(n+1)}{2}$

- i. $n = 1, P(1)$ est vraie $1 = \frac{1(2)}{2}$
- ii. On suppose que $1 + 2 + 3 + \dots + n = \frac{n(n+1)}{2}$ est vrai
- iii. On montre que $1 + 2 + \dots + n + 1 = \frac{(n+1)(n+2)}{2}$ est vraie ainsi P est vraie à l'ordre $n+1$ alors $\forall n \in \mathbb{N}^*$:

$$1 + 2 + \dots + n = \frac{n(n+1)}{2}$$

Remarque: Le theoreme precedent est le premier principe d'induction

1.4 Induction et recurrence

Induction et recurrence sont deux concepts intimement liés, deux facettes d'une meme idee.

Dans l'usage courant, les deux mots sont employés de facons interchangeable. En tout etat de cause, la frontiere entre ces notions est floue; c'est pour cela qu'on a pris le parti de traiter les deux concepts de manieres correllées.

Formellement l'induction est une technique de preuve mathématique pour demontrer des theoremes sur les nombres naturels ou sur des structures infinies munies d'un ordre bien fondés (Souvent des steuctures definies recursivement). D'un autre coté, la recursion est une technique de definition et construction d'objets mathématiques (fonctions, ensembles, ...).

Ce que les deux concepts ont en commun, c'est l'idée d'etudier les propriétés d'un objet (par exemple une propriété d'un entier) en les reduisant aux propriétés d'objets plus petits (par exemple les propriiétés de $n-1$)

1.4.1 Induction

Dans sa forme la plus simple, le principe d'induction est une technique de preuve qui permet de prouver qu'un predicat $P(n)$ est vrai pour tous les entiers $(\forall n, P(n))$.

Le raisonnement par recurrence est la forme la plus simple de la preuve par induction

Induction forte

L'induction forte est une autre forme d'induction couramment utilise. Dans une preuve par induction forte, il suffit de montrer que: $\forall k < n P(k) \Rightarrow P(n)$

Exemple: Montrer que tout nombre entier naturel $n \geq 2$ possede un diviseur premier *Preuve:* Soit $n > 2$, on suppose que tous les nombre entiers $2 \leq k \leq n$ possede un diviseur premier(hypothese de recurrence) et on va montrer que $n + 1$ possede un diviseur premier.

Il existe 2 cas:

- $n + 1$ est premier, alors il possède un diviseur premier, lui-même
- $n + 1$ est composé, alors $\exists$ un entier $m > 2$ et $m \leq n$ qui divise $n + 1$. Alors, par hypothèse de récurrence, m possède un diviseur premier, qui est aussi un diviseur de $n + 1$

Generalisation a d'autres ensemble que les entiers naturels

L'application du principe d'induction n'est pas restreinte aux entiers, en effet, il peut être appliqué à tout ordre bien fondé, à savoir qu'il n'existe pas de chaîne descendante infinie, garantissant que toute preuve par induction se termine après un nombre fini d'étapes. Ce type d'induction est parfois appelée induction structurale. Il est spécialement utile par raisonnement sur des structures de données tels que les arbres, les listes,...

1.4.2 Recursion

La récursion est une technique de définition d'objets mathématiques. Comme pour l'induction, on définit récursivement (ou inductivement) une famille d'objets en deux étapes:

- On définit un nombre fini de définitions explicites d'objets de base
- On définit tous les autres objets en fonction d'objets plus petits définis précédemment

Formellement une fonction peut être définie sur tout ensemble muni d'un ordre bien fondé ou de façon équivalente en présence d'un axiome d'induction

Theorem 2. *Soit A un ensemble d'une relation d'ordre bien fondée et soit B un autre ensemble quelconque. Une fonction (récursive) $f : A \rightarrow B$ est bien définie (ou cohérente ou bien fondée). Si $\forall a \in A$, la définition de f ne fait pas intervenir que les valeurs $f(a')$ pour $a < a'$*

Travaux Dirigés

Exercice I:

Prouver que pour chaque entier naturel $n > 1$:

- i. n est un diviseur premier
- ii. si n n'est pas premier, il a un diviseur premier p tel que $p \leq \sqrt{n}$
- iii. il existe un nombre premier strictement plus grand que n

Exercice II:

- i. Prouver la proposition suivante:

$$\forall n \in \mathbb{N} : \sum_{i=0}^n i = \frac{n(n+1)}{2}$$

- ii. Prouver que $1 + 3 + 5 + \dots + (2n - 1) = n^2$

- iii. Prouver la proposition suivante:

$$\forall n \in \mathbb{N} : \sum_{i=0}^n i^2 = \frac{n(n+1)(2n+1)}{6}$$

Exercice

Ecrire le code python de la fonction factorielle iterative et recursive

Chapter 2

RECURSIVITES ET INDUCTIONS

2.1 Methode de raisonnement

Pour montrer que $P \implies Q$ est vrai, on peut utiliser plusieurs principes de raisonnement empruntés à la logique mathématique. Ces principes interviennent dans l'élaboration d'algorithmes divers

2.1.1 Methode de raisonnement direct

On suppose que P est vrai et on démontre Q l'est aussi

Exemple: Montrons que $\forall n \in \mathcal{N}, n \text{ est pair} \implies n^2 \text{ est pair}$

2.1.2 Raisonnement par récurrence sur $\mathbb{N}$

Theorem 3. *Pour montrer que $P(n): \forall n \in \mathbb{N} \ n \geq n_0, P_n(x)$ est vrai on suit les étapes suivantes:*

- i. *On montre que $P(n_0)$ est vrai*
- ii. *On suppose que $P(n)$ est vraie à l'ordre n*
- iii. *On démontre que $P(n+1)$ est vraie à l'ordre $n+1$ alors P est vrai $\forall n \geq n_0$*

Exemple: Montrer que $\forall n \in \mathbb{N}^*$,

$$1 + 2 + 3 + \dots + n = \frac{n(n+1)}{2}$$

Remarque: Le théorème précédent est le premier principe d'induction

2.2 Inductions et recurrences

Inductions et recurrences sont deux concepts intimement liés, deux facettes d'une même idée. Dans l'usage courant, les deux mots sont employés de façons interchangeables et en tout cas avec une frontière assez floue. Formellement l'induction est une technique de preuve mathématique employée pour démontrer des théorèmes sur les nombres naturels ou sur d'autres structures infinies munies d'un ordre bien fondé (souvent des structures définies récursivement)

D'un autre côté la récursion est une technique de définition et construction d'objets mathématiques (fonctions, ensembles...).

Ce que les deux concepts ont en commun, c'est l'idée d'étudier les propriétés d'un objet (Par exemple une propriété d'un entier n) en les réduisant aux propriétés d'objets plus petits (par exemple les propriétés de $n - 1$)

2.2.1 Principe général d'une fonction inductive

Dans sa forme la plus simple, **le principe d'induction** est une technique de preuve qui permet de prouver qu'un prédicat $P(n)$ est vrai pour tous les entiers ($\forall n, P(n)$) **Exemple** Montrer que pour tout entier n : $10^n - 1$ est divisible par 9. Donc ici la propriété à démontrer est :

$P(n) : 10^n - 1$ est divisible par n

Une preuve par induction procède en deux étapes :

- i. On démontre que le prédicat est vrai pour un nombre fini de cas initiaux
- ii. On démontre que si le prédicat est vrai pour un nombre de cas quelconque, alors il est vrai aussi pour le cas suivant

On peut faire remonter son origine aux mathématiques classiques, par exemple l'argument d'Euclide prouvant qu'il existe une infinité de nombres premiers est une forme implicite d'induction. Pascal a été l'un des premiers mathématiciens à en donner une définition ex-

plicité, mais une formalisation rigoureuse n'est apparue qu'au 19e siècle dans les travaux de Peano, Boole, etc.

2.2.2 Preuve par Induction

Soit $P(n)$ un prédicat portant sur une variable libre représentant un entier. Le but d'une preuve par induction est de montrer le prédicat

$$\forall n. P(n)$$

Il existe plusieurs formes équivalentes du principe d'induction. La plus commune, aussi appelée induction faible ou simple, procède en deux étapes :

- Un cas de base (aussi appelé initialisation), où on démontre le prédicat , où est le plus petit entier pour lequel la propriété est supposée vraie.
- Un cas récursif (ou hérédité, ou pas inductif), où on démontre le prédicat $P(n) \implies P(n+1)$

De ces deux lemmes, le principe d'induction déduit $\forall n. P(n)$. En effet on sait par la base que $P(0)$, puis en utilisant l'hérédité on déduit $P(1)$ de $P(0)$, puis $P(2)$ de $P(1)$, et ainsi de suite.

On peut donner une preuve plus rigoureuse.

Preuve par l'absurde On suppose le contraire, c'est-à-dire que $P(n)$ n'est pas vrai pour tout n . On note $X = \{k \in \mathbb{N} \mid P(k) \text{ est fausse} \}$. On a supposé que X est non-vide, il admet donc un plus petit élément . D'après la première condition $m \neq 0$ et donc $m-1$ est un entier naturel et $P(m-1)$ est un entier naturel et $P(m-1)$ vrai par la définition de X . On obtient une contradiction avec la deuxième propriété appliquée à l'entier $m-1$.

Induction forte

L'induction forte est une autre forme d'induction couramment utilisée. Dans une preuve par induction forte il suffit de démontrer que

$\forall k < n. P(k)$ implique $P(n)$

pour déduire la vérité du prédicat $\forall n. P(n)$. Cette forme d'induction est équivalente à l'induction faible, en effet prouver $P(n)$ par induction forte revient à prouver $\forall k < n. P(k)$ par induction faible.

Note Le cas de base de l'induction forte est implicitement contenu dans sa définition. En effet lorsque $n = 0$, prouver $(\forall k < 0. P(k)) \implies P(0)$ revient à prouver $P(0)$, , car la quantification sur la gauche est vide (et donc toujours vraie).

Exercice Montrer que tout entier naturel $n > 2$ possède un diviseur premier.

Chapter 3

MOTS ET LANGAGES

Introduction

La théorie des langages formels trouve son origine dans l'étude des langues naturelles.

La description d'une langue naturelle est traditionnellement appelée **une grammaire**; elle doit indiquer comment les phrases d'une langue sont composées d'éléments, comment ces éléments forment une plus grandes unités, et comment ces unités sont liées dans phrase.

D'un point de vue mathématique, les grammaires sont des systèmes formels, comme les machines de Turing, les programmes informatiques, la logique prépositionnelle, les théories de l'inférence, etc. Des lors les expressions **mots**, **phrase**, **langages** ne doivent pas être utilisés dans leur sens linguistique.

3.1 Définition préliminaires

Definition 12. *Un alphabet est un ensemble fini désigné par un caractère qui est la plus part du temps de l'alphabet grecque.*

Ainsi $\Sigma = \{a, b, c\}$, $\sigma = \{0, 1\}$, $\phi = \{\leftarrow, \rightarrow, \downarrow, \uparrow\}$ sont des alphabets.

Les éléments de l'alphabet sont appelées des lettres ou des symboles

Exemple: Le biologiste intéressé par les groupes sanguins aura l'alphabet suivant $\sigma = \{A, O, B, O...\}$

Definition 13. *Soit Σ un alphabet. Un mot sur Σ est une suite finie (et ordonnée) de symboles. Par exemple, *abba* et *ab* sont deux mots sur $\Sigma = \{a, b, c\}$. La longueur d'un mot w est le nombre de symboles constituant le mot; on note $|w|$ cette longueur*

Exemple: $|abbac| = 5$ et $|ab| = 2$

On ϵ l'unique mot de longueur 0 qui correspond à la suite vide. ϵ s'appelle le mot vide.

On appelle Σ^* l'ensemble des mots sur Σ privé de ϵ

$$\{a, b, c\}^* = \{a, b, c, aa, ab, ac, ba, bb, bc, abc.. \}$$

Definition 14. Si σ est une lettre de l'alphabet Σ , pour tout $w = w_1, \dots, w_k \in \Sigma^*$, on dénote par

$$|w|_\sigma = \#\{i \in \{1, \dots, k\} | w_i = \sigma\}, \text{ le nombre de lettre } \sigma_i \text{ apparaissant dans le mot } w$$

$$|abbac|_a = 2 \text{ et } |abbac|_c = 1$$

Definition 15. Grammaire: une grammaire $G = (V_N, V_T, P, S)$ composé d'un vocabulaire non-terminal V_N , un vocabulaire terminal V_T , une production P et un symbole de depart S avec :

- i. V_N, V_T , et P sont des ensembles fini non-vide
- ii. $V_N \cap V_T = \emptyset$
- iii. $P \in V^+ \times V^*$
- iv. $S \in V_N$

Une phrase generée par G est tout element s de V_T^* pour lequel $S \Rightarrow s$ ie tout mot derivé de S par la production P

Ajoutez un exemple pour la compréhension

Definition 16. Langage: Un langage sur Σ est un ensemble fini ou infini de mots de Σ . En d'autres termes un langage est une partie de Σ^* .

On distingue en particulier le langage vide $\emptyset$

Le langage $L(G)$ généré par G est l'ensemble des phrases généré par P **Exemple:** Considerons l'alphabet des nombres en base hexadecimales. On peut extraire plusieurs sous partitions du langage formé par cet alphabet

Definition 17. Soit L et $M \in \Sigma^*$, deux langages. La concatenation des langages L et M est le langage $LM = \{uv | u \in L, v \in M\}$

En particulier:

pour $n > 1$ on définit la puissance n^{ieme} de L par $L^n = w_1...w_n | \forall i \in \{1, ..., n\}, w_i \in L$ et on note $L^0 = \{\epsilon\}$

Exemple pour $L = \{a, ab, ba, ac\}$ alors $L^2 = \{aa, aab, abba, aac....\}$

La concatenation de langages est une operation associative. Elle possède $\{\epsilon\}$ pour element neutre, $\emptyset$ pour élément absorbant. La concatenation est distributive à droite comme à gauche pour l'union, i.e Si L_1, L_2 et L_3 sont des langages,

- $L_1(L_2L_3) = (L_1L_2)L_3$
- $L_1\{\epsilon\} = \{\epsilon\}L_1 = L_1$
- $L_1\emptyset = \emptyset L_1 = \emptyset$
- $L_1(L_2 \cup L_3) = (L_1L_2) \cup (L_1L_3)$

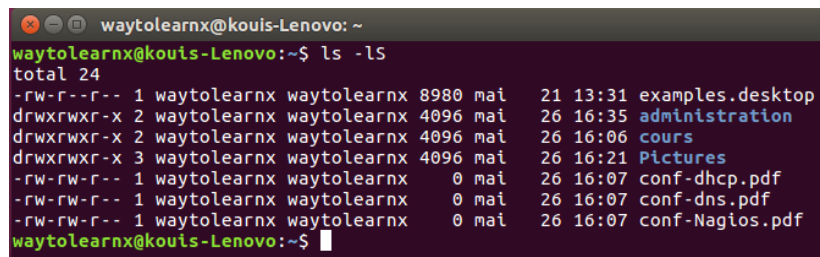
Definition 18. Soit $L \in \Sigma^*$, l'étoile de Kleene de L est définie par $L^* = \cup_{i \geq 0} L^i$

N.B: Les mots de L^* sont exactement les mots obtenus en concaténant un nombre arbitraire de mots de L .

On rencontre les rotations: $L^+ = \cup_{i \geq 1} L^i$ et $\Sigma^+ = \Sigma \setminus \{\epsilon\}$

3.2 Expressions régulières et langages associés

La notion d'expression régulière est d'usage fréquent en informatique. Le meilleur exemple reste celui d'un repertoire contenant deux fichiers



```
waytolearnx@kouis-Lenovo: ~  
waytolearnx@kouis-Lenovo:~$ ls -ls  
total 24  
-rw-r--r-- 1 waytolearnx waytolearnx 8980 mai 21 13:31 examples.desktop  
drwxrwxr-x 2 waytolearnx waytolearnx 4096 mai 26 16:35 administration  
drwxrwxr-x 2 waytolearnx waytolearnx 4096 mai 26 16:06 cours  
drwxrwxr-x 3 waytolearnx waytolearnx 4096 mai 26 16:21 Pictures  
-rw-rw-r-- 1 waytolearnx waytolearnx 0 mai 26 16:07 conf-dhcp.pdf  
-rw-rw-r-- 1 waytolearnx waytolearnx 0 mai 26 16:07 conf-dns.pdf  
-rw-rw-r-- 1 waytolearnx waytolearnx 0 mai 26 16:07 conf-Nagios.pdf  
waytolearnx@kouis-Lenovo:~$
```

Figure 3.1

L'utilisateur aura par exemple recours à une commande comme $ls^*.jpg$ pour afficher uniquement les images au format "jpg" ou encore m^* pour effacer tous les fichiers relatifs en mémoire.

cette section s'applique en pratique à ce genre de critères.

Definition 19. Soit Σ un alphabet. Supposons que $o, e, +, ., (,)^*$ sont des symboles n'appartenant pas à Σ . L'ensemble R_Σ des expressions régulières sur Σ est défini récursivement par:

- o et e appartiennent à R_Σ
- Pour tout $\sigma \in \Sigma$, σ appartient à R_Σ
- Si ϕ et ψ appartiennent à R_Σ alors
 - $(\phi + \psi)$ appartient à R_Σ
 - $(\phi.\psi)$ appartient à R_Σ
 - ϕ^* appartient à R_Σ

Exemple1: Si $\Sigma = \{a, b\}$ voici quelques exemples d'expressions régulières.

- $\alpha_1 = (a + (a.b))$
- $\alpha_2 = (((a.b).a) + b^*)^*$
- $\alpha_3 = ((a + b)^*. (a.b))$

A une expression régulière, on associe un langage grâce à l'application $\mathcal{L} : R_\Sigma \rightarrow 2^{\Sigma^*}$

La notation 2^{Σ^*} désigne l'ensemble des parties de Σ^* i.e l'ensemble des langages sur Σ . On trouve parfois la notation $\mathcal{P}(\Sigma^*)$ par

- $\mathcal{L}(0) = \emptyset, \mathcal{L}(e) = \{\epsilon\}$
- Si $\sigma \in \Sigma$, alors $\mathcal{L}(\sigma) = \{\sigma\}$
- Si ϕ et ψ sont des expressions régulières,
 - $\mathcal{L}(\phi + \psi) = \mathcal{L}(\phi) \cup \mathcal{L}(\psi)$

$$- \mathcal{L}(\phi\psi) = \mathcal{L}(\phi)\mathcal{L}(\psi)$$

$$- \mathcal{L}(\phi^*) = (\mathcal{L}(\phi))^*$$

Exemple2 De l'exemple 1, on a:

- $\mathcal{L}(\alpha_1) = \{\epsilon, ab\}$
- $\mathcal{L}(\alpha_1) = (\{aba\} \cup \{b\}^*)^*$
- $\mathcal{L}(\alpha_3) = \{a, b\}^*ab$

Definition 20. Un langage L sur Σ est **regulier** s'il existe une expression régulière $\phi \in R_\Sigma$ telle que:

$$L = \mathcal{L}(\phi)$$

Si ϕ et $\psi \in R_\Sigma$ telles que $\mathcal{L}(\phi) = \mathcal{L}(\psi)$

Remarque: Dans la suite

- On s'autorisera à confondre une expression régulière au langage qu'elle représente
- On omettra les parenthèses ou autres symboles superflus. Par exemple,

$$((b^*.a).(b^*.a))^* = (b^*ab^*b^*a)^*$$

$$((b^*a).(b^*a).b^*)^* = (b^*ab^*ab^*)^*$$

$$(((b^*a).(b^*a)).b^*)^* = (b^*ab^*ab^*)^*$$

La dernière expression régulière représente le langage formé par des mots $\{a, b\}$. On se convainc aisément que ce langage est aussi représenté par $b^*(ab^*ab^*)^*$

Proposition: L'ensemble $\mathcal{L}(R_\Sigma)$ des langages réguliers sur Σ est la plus petite famille de langages contenant le langage vide, les langages $\{\sigma\}$ réduits à une lettre ($\sigma \in \Sigma$) et qui est stable pour les opérations d'union, de concatenation et d'étoile de Kleene

Chapter 4

AUTOMATES

Les automates dans ce chapitre sont des machines permettant de reconnaître les langages vus dans le chapitre 2. En d'autres termes, l'ensemble des langages acceptés par automates fini coïncide avec l'ensemble des langages réguliers.

4.1 Automates finis déterministes AFD

Definition 21. *Un AFD est la donnée d'un quintuple*

$$\mathcal{A} = (Q, q_0, F, \Sigma, \delta)$$

où

- Q est un ensemble fini dont les éléments sont les états de $\mathcal{A}$,
- $F \in Q$ est un état privilégié appelé état initial,
- Σ est l'ensemble des états finaux
- $\delta : Q \times \Sigma \rightarrow Q$ est la fonction de transition de $\mathcal{A}$

Nous supposons que δ est une fonction totale, i.e que δ est défini pour tout couple $(q, \sigma) \in Q \times \Sigma$ (On parle alors d'AFD complet)

Nous représentons un AFD $\mathcal{A}$ de la manière suivante:

- Les états de $\mathcal{A}$, représentés par des cercles, sont les sommets d'un graphe orienté
- Si $\delta(q, \sigma) = q'$; $q, q' \in Q, \sigma \in \Sigma$ alors on trace Un arc orienté de q vers q' et de label σ
 $q \xrightarrow{\sigma} q'$
- Chaque état final est représenté par deux cercles concentriques
- L'état initial est désigné par une flèche entrante sans label

- Si $\delta(q, \sigma) = q'$ et $\delta(q, \sigma') = q'$, on dessinera $q \xrightarrow{\sigma, \sigma'} q'$, Ceci s'adapte aisement à plus de deux lettres

Exemple1 : L'automate $\mathcal{A} = (Q, q_0, F, \Sigma, \delta)$ où $Q = \{1, 2, 3\}$, $q_0 = 1$, $F = \{1, 2\}$, $\Sigma = \{a, b\}$ où la fonction de transition est donnée par la table

δ	a	b
1	1	2
2	1	3
3	3	2

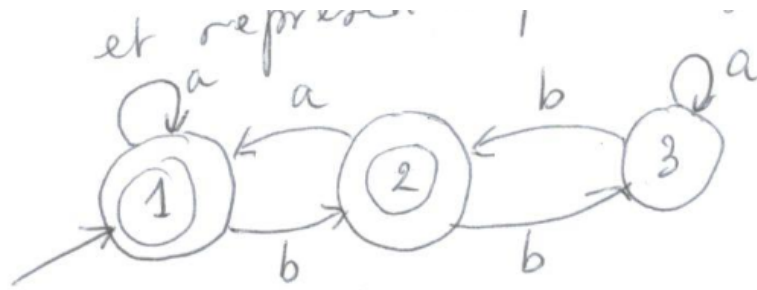


Figure 4.1: Automate 1

Definition 22. Soit $\mathcal{A} = (Q, q_0, F, \Sigma, \delta)$ un AFD. On étend naturellement la fonction de transition à $Q \times \Sigma^*$ de la manière suivante: $\delta(q, \epsilon) = q$ et $\delta(q, \sigma w) = \delta(\delta(q, \sigma), w)$; $\sigma \in \Sigma, w \in \Sigma^*$

Le langage accepté par $\mathcal{A}$ est alors $\mathcal{L}(\mathcal{A}) = \{w \in \Sigma^* | \delta(q_0, w) \in F\}$

Definition 23. A tout mot $w \in \Sigma^*$ correspond une unique suite d'états de $\mathcal{A}$ correspondant au chemin parcouru lors de la lecture de w dans $\mathcal{A}$.

Ainsi si $w = w_0 \dots w_k$ avec $w_i \in \Sigma$ alors l'exécution de w sur $\mathcal{A}$ est $(q_0, q_0 w_0, q_0 w_0 w_1, \dots, q_0 w_0 \dots w_k)$

4.2 Automates finis non-deterministes(AFND)

Definition 24. Un automate fini non-deterministe(AFND) est la données d'un quintuple $\mathcal{A} = (Q, I, F, \Sigma)$ où:

- Q est un ensemble fini dont les éléments sont les états de $\mathcal{A}$

- ii. $I \subset Q$ est l'ensemble des états initiaux
- iii. $F \subset Q$ est l'ensemble des états finaux
- iv. Σ est l'alphabet de l'automate,
- v. $\Delta \subset Q \times \Sigma^* \times Q$ est une relation de transition (qu'on supposera)

Il y'a plusieurs différences entre un AFD et un AFND:

- Dans le cas non-deterministe, il est possible d'avoir plus d'un état initial; les labels ne sont plus nécessairement des lettres mais bien des mots de Σ^* et enfin, on n'a plus *une fonction de transition* mais **une relation de transition**
- Pour représenter les AFND, nous utiliserons les memes conventions que pour les AFD

Exemple1

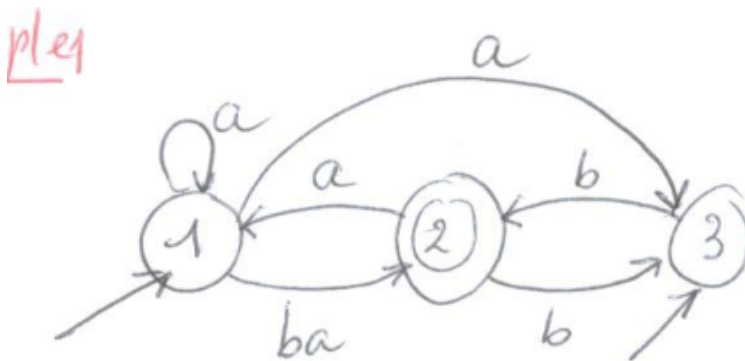


Figure 4.2: AFND

Dans cet AFND

$$I = \{1, 3\}, F = \{2\}$$

$$Q = \{1, 2, 3\}, \Sigma^* = \{a, b, ba\}$$

$$\Delta = \{(1, a, 1), (1, a, 3), (1, ba, 2), (2, b, 3), (3, b, 2), (2, a, 1)\}$$

Definition 25. Un mot $w = w_1 \dots w_k$ est accepté par un AFND $\mathcal{A} = (Q, I, F, \Sigma, \Delta)$ s'il existe $q_0 \in I, l \in \mathbb{N} \setminus \{0\}, v = v_1 \dots v_k \in \Sigma^*, q_1 \dots q_l \in Q$ telle que $(q_0, v_1, q_2), (q_1, v_2, q_2), \dots, (q_{l-1}, v_l, q_l) \in \Delta$

$w = v_1 \dots v_l$ et $q_l \in F$

Exemple2: Dans l'exemple1, le mot ab est accepté car $1 \in I, (1, a, 3) \in \Delta, (3, b, 2) \in \Delta$ et $2 \in F$. Ceci se note schématiquement $1 \xrightarrow{a} 3 \xrightarrow{b} 2$

A un mot il peut correspondre plusieurs chemins:

- i. $3 \xrightarrow{b} 2 \xrightarrow{a} 1 \xrightarrow{a} 1$
- ii. $3 \xrightarrow{b} 2 \xrightarrow{a} 1 \xrightarrow{a} 3$
- iii. $1 \xrightarrow{ba} 2 \xrightarrow{a} 1$

Ce sont les trois seules possibilités partant d'un état initial d'obtenir le mot baa . ce mot n'est donc pas accepté par l'automate **Rémarque** Dans un AFND, on peut avoir des transitions vides du type $(q, \epsilon, q') \in \Delta$.

On parle alors de ϵ -transitions.

En particulier, on suppose implicitement que pour état q d'un AFND, on $(q, \epsilon, q) \in \Delta$

Exemple3

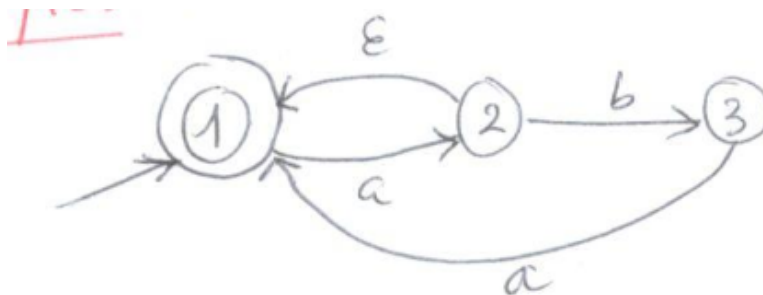


Figure 4.3: ϵ -transition

pour le mot a , on a le chemin $1 \xrightarrow{a} 2 \xrightarrow{\epsilon} 1 \in F$ Remarquons qu'ici il n'est pas possible de lire des mots commençant par b . Donc contrairement à la situation déterministe où chaque mot correspondait exactement à une exécution, ici on peut avoir pour un mot donné plus d'un chemin dans le graphe, voir aucun

Definition 26. Un AFND $\mathcal{A} = (Q, I, F, \Sigma, \Delta)$ est qualifié d'élémentaire si pour tout $(q, w, p') \in \Delta$, $|w| \leq 1$

Lemme: Tout langage accepté par un AFND est accepté par un AFND élémentaire

Exemple 4: Application de la méthode de construction contenue dans la preuve de ce lemme à l'exemple de l'AFND qui suit:

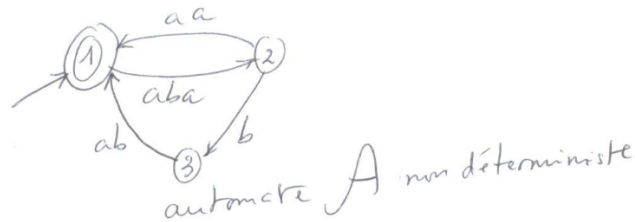


Figure 4.4: Automate non-deterministe

Démonstration du lemme:

Soit $\mathcal{A} = (Q, I, F, \Sigma, \Delta)$ un AFND. S'il n'est pas élémentaire, il existe au moins un mot $w = w_1 \dots w_k (k \geq 2, w_i \in \Sigma)$ et des états q, q' tels que $(q, w, q') \in \Delta$.

Considérons de nouveaux états $q_1, \dots, q_{k-1}$ n'appartenant pas à Q . Il est clair que l'automate $\mathcal{B} = (Q', I, F, \Sigma, \Delta')$

où $Q' = Q \cup \{q_1, \dots, q_{k-1}\}$ et

$\Delta' = \Delta \setminus \{(q, w, q^{k-1})\} \cup \{(q, w_1, q_1), (q_1, w_2, q_2), \dots, (q_{k-2}, w_{k-1}, q_{k-1}), (q_{k-1}, w_k, q')\}$ est telle que $L(A) = L(B)$. En répétant cette procédure pour chaque transition $(q, w, q') \in \Delta$ tel que $|w| \geq 1$, on obtient (en un nombre fini d'étapes) un automate élémentaire acceptant le même langage.

En revenant à l'exemple 4 on obtient alors un automate élémentaire équivalent à $\mathcal{A}$ représenté ci-dessous où les états supplémentaires ne portent pas de numéro

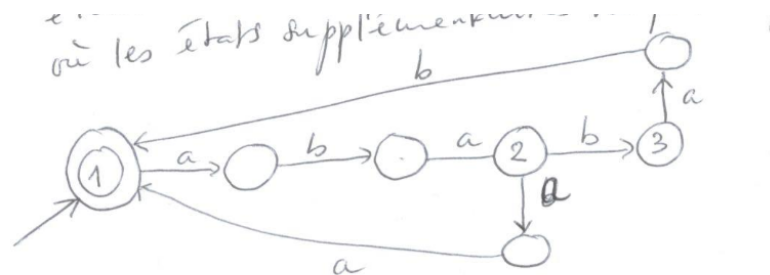


Figure 4.5: Automate élémentaires

Remarque: Soit $\mathcal{A} = (Q, I, F, \Sigma, \Delta)$ un AFND. Si $R \subset Q$ et $w \in \Sigma^*$, on note:

$R.w$ l'ensemble des états atteints à partir des états de R en lisant w . Par exemple, avec l'automate de l'exemple 3 $\{1\}.a = \{1, 2\}$ car $1 \xrightarrow{a} 2$ et $1 \xrightarrow{a} 2 \xrightarrow{\epsilon} 1$.

On a aussi pour cet automate $\{1\}.b = \emptyset$. Dans le cas où $w = \epsilon$, on a toujours $R.\epsilon \supseteq R$.

$R.\epsilon \supseteq R$ est l'ensemble des états atteints depuis les états de R sans lire de lettres.

$R.L = \cup_{w \in L} R.w$ où L est un langage sur Σ .

Proposition(Rabin & scott): Tout langage accepté par un AFND est accepté par un AFD

Demonstration

D'après le lemme vu plus haut, on peut supposer disposer d'un AFND élémentaire $\mathcal{A} = (Q, I, F, \Sigma, \Delta)$. Considerons l'AFD $\mathcal{B} = (2^Q, Q_0, \mathcal{F}, \Sigma, \beta)$ ayant comme ensemble d'états, l'ensemble des parties de Q et tel que:

- i. L'état initial Q_0 est égale à $I.\epsilon$, i.e à l'ensemble des états de $\mathcal{A}$ atteints à partir des états initiaux sans consommer des lettres;
- ii. L'ensemble des états finaux est $\mathcal{F} = \{G \subset Q | G \cap F \neq \emptyset\}$ i.e les états finaux de $\mathcal{B}$ sont les parties de Q contenant au moins un état final
- iii. Si $G \subset Q$ est un état de $\mathcal{B}$ et w un mot non-vide sur Σ , alors on définit la fonction de transition de $\mathcal{B}$ par $\mathcal{B}(G, w) = G.w$. De plus, on pose $\mathcal{B}(G, \epsilon) = G$.

Pour montrer qu'il s'agit d'un AFD, il suffit de montrer que β est une fonction de transition, i.e que pour tous $u, v \in \Sigma^*$, $\beta(G, u.v) = \beta(\beta(G, u), v)$.

Si au moins un des mots u ou v est nul, la constitution est immédiate. Nous devons montrer que $G.uv = (G.u).v$. Il est clair que $(G.u).v \subset G.uv$. L'autre inclusion résulte du fait que l'automate est élémentaire.

N.B La propriété d'automate élémentaire est nécessaire comme illustré ci-dessous:

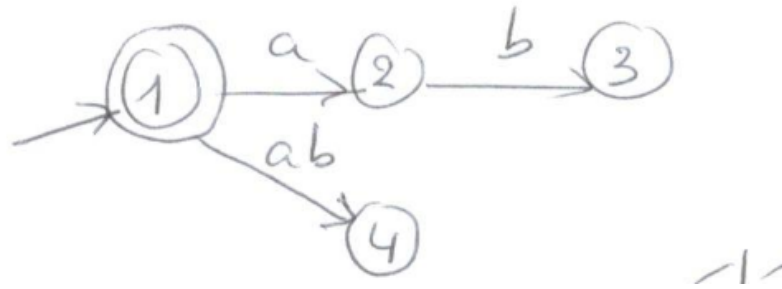


Figure 4.6: Automate non-élémentaire

On a $\{2\}, (\{1\}.a).b = \{2\}.b = 3$ donc $\{1\}.ab \notin (\{1\}.a).b$ puisque $\{1\}.ab = \{4\}$

Il reste à montrer que $\mathcal{L}(A) = \mathcal{L}(B)$. Soit $w \in \mathcal{L}(A)$. Cela signifie qu'il existe dans $\mathcal{A}$ un chemin de label w débutant dans un état initial de I (et même dans un état de $I.\epsilon$) et aboutissant dans un état final de F . En d'autres termes, par définition de Q_0 , $Q_0.w \cap F \neq \emptyset$ et $\beta(Q_0, w) \in \mathcal{F}$

Soit $w \in \mathcal{L}(\beta)$. Dans β , le chemin de label w débutant dans Q_0 aboutit dans un état final de $\mathcal{F}$ i.e $\beta(Q_0, w) \in \mathcal{F}$, donc $(I; \epsilon).w \cap \mathcal{F} \neq \emptyset$ Ce qui signifie que w est accepté par $\mathcal{A}$

Méthode permettant de rechercher un AFD acceptant exactement le même langage qu'un AFND donné: Seules les états peuvent être atteints depuis Q_0 (l'ensemble des états de $\mathcal{A}$ atteints à partir des états initiaux sans consommer des lettres, $I.\epsilon$) méritent d'être considérés. Q_0 correspond à l'état initial de l'AFD.

Exemple: Considérons l'AFND suivant

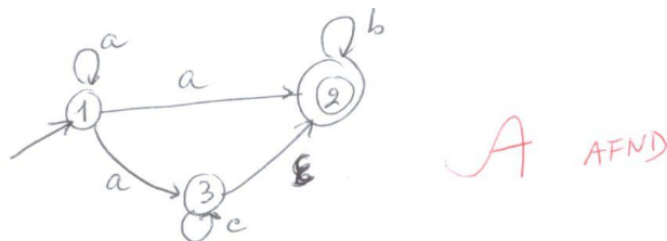


Figure 4.7: AFND $\mathcal{A}$

On remarque qu'il est élémentaire. S'il ne l'était pas il faudrait tout d'abord le rendre élémentaire

On construit le tableau suivant de proche en proche en l'initialisant avec $I.\epsilon$ qui est tel que ici

$$\{1\}.\epsilon = \{1\}$$

A chaque étape, pour un sous-ensemble X d'états non encore traité, on détermine les valeurs de $X.\sigma$ pour tout $\sigma \in \Sigma$. La construction se termine une fois que tous les sous-ensembles d'états apparaissant ont été pris en compte

X	X.a	X.b	X.c
$\{1\}$	$\{1,2,3\}$	$\emptyset$	$\emptyset$
$\{1,2,3\}$	$\{1,2,3\}$	$\{2\}$	$\{2,3\}$
$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$
$\{2\}$	$\emptyset$	$\{2\}$	$\emptyset$
$\{2,3\}$	$\emptyset$	$\{2\}$	$\{2,3\}$

La construction d'un tel tableau peut être facilitée en établissant au préalable la table de transition de l'automate. Ici, pour $\mathcal{A}$, cette table est:

q	q.a	q.b	q.c
1	$\{1, 2, 3\}$		
2		2	
3		2	2, 3

Si on pose $A = \{1\}$, $B = \{1, 2, 3\}$, $C = \emptyset$, $D = \{2\}$, $E = \{2, 3\}$ alors on a l'automate représenté ci-dessous

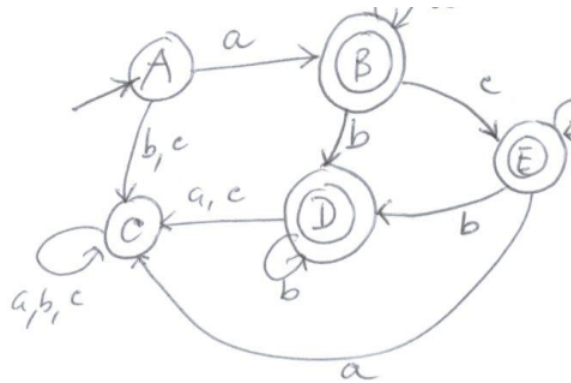


Figure 4.8: AFD

Les états finaux de cet AFD sont les sous-ensembles d'états de $\mathcal{A}$ qui contiennent 2 (le seul état final de $\mathcal{A}$)

Chapter 5

Travaux Dirigés

5.1 Exercice I: Logiques et Fonctions booléennes

I.1 On considère la fonction booléenne suivante:

$$f(x, y, z) = (x \wedge \neg y \wedge \neg z) \vee (\neg x \wedge y \wedge \neg z) \vee (\neg x \wedge \neg y \wedge z)$$

Donnez sa table de vérité. Que fait cette fonction? Donner une expression booléenne plus simple pour cette fonction

I.2 Montrez de deux manières l'égalité suivante:

$$(x \wedge y) \vee (\neg y \wedge z) = (x \vee \neg y) \wedge (y \vee z) \quad (5.1.1)$$

5.2 Exercice II: Automates

I.1 On considère l'alphabet $A = a, b, c$. Soit le mot $u = abbc$.

- i. Ecrire une expression rationnelle pour le langage de tous les mots qui commencent par u (par exemple, $abbcabbcc$ commence par u).
- ii. Ecrire une expression rationnelle pour le langage de tous les mots qui terminent par u (par exemple, $babbababcbcc$ termine par u).
- iii. Ecrire une expression rationnelle pour le langage de tous les mots qui contiennent u (par exemple, $bbabcbabbcbcc$ contient u).

I.2 On considère l'automate $\mathcal{A}$ suivant qui reconnaît $\mathcal{L}(\mathcal{A})$:

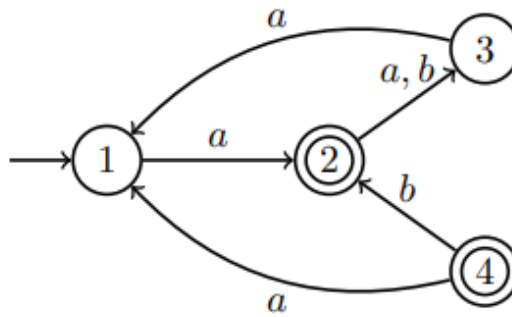


Figure 5.1

- i. Est-il déterministe? Est-il complet?
- ii. Donnez un mot de longueur 4 qui est reconnu par $\mathcal{A}$ et un qui n'est pas reconnu.
- iii. À quoi sert l'état 4
- iv. Proposez un automate qui reconnaît le complémentaire de $\mathcal{L}(\mathcal{A})$

I.3 On se place sur $\mathcal{A} = 0, 1$.

- i. Donnez un automate déterministe et complet $\mathcal{A}_{\in}$ qui reconnaît les mots qui sont la représentation binaire d'un nombre pair.
- ii. Donnez un automate déterministe et complet $\mathcal{A}_{\ni}$ qui reconnaît les mots qui sont la représentation binaire divisible par 3.
- iii. Calculez l'automate produit de $\mathcal{A}_{\in}$ et $\mathcal{A}_{\ni}$. Quel langage reconnaît-il ?